Meridian — Technical Document

Languages, frontend/backend stack, database connection aur architecture — manager ke sawalon ke jawab.

CONTENTS

1. 1. Ek page mein poora project
2. 2. Kaunsi languages use hui hain
3. 3. Frontend — kya use hua aur kyun
4. 4. Backend — kya use hua aur kyun
5. 5. Database — SQLite aur connection kaise hota hai
6. 6. Architecture — request ka safar
7. 7. API — 39 endpoints
8. 8. Authentication aur security
9. 9. Roles aur permissions
10. 10. Core workflow — technical summary
11. 11. AI integration
12. 12. Reports aur insights
13. 13. Project structure
14. 14. Dependencies — sirf 12 direct packages
15. 15. Build, run, test aur deploy
16. 16. Testing aur quality
17. 17. Important design decisions
18. 18. Limitations aur future scope
19. 19. Manager ke common sawal
20. 20. Glossary — technical shabd simple mein
21. 21. Ek line mein

> **Ye document kis ke liye hai?** Manager, reviewer ya kisi bhi technical stakeholder ke liye jo poochh sakta hai: "Project kaunsi language mein bana hai, frontend/backend mein kya use hua, aur database se connection kaise hota hai?" Har jawab simple bhasha mein, actual code ke hisaab se diya gaya hai.

---

1. Ek page mein poora project

Sawal	Jawab
Frontend	**React 19 + TypeScript + Vite**
Backend	**Node.js 24 + Express 5** (JavaScript)
Database	**SQLite** — Node ka built-in `node:sqlite`
Database connection	Koi server nahi; ek local file (`meridian.db`) directly open hoti hai
ORM	Nahi — **raw SQL + prepared statements**
Authentication	scrypt password hashing + HMAC-SHA256 signed token
AI	OpenAI-compatible API (`gpt-4o-mini` default) + built-in fallback
Styling	Hand-written CSS (Tailwind/Bootstrap nahi)
Hosting	Ek Docker service jo UI + API dono serve karta hai
Tests	Node ka built-in test runner — **12 scenarios**
Application code	Approx **17,350 lines**, ~53 source files
Database tables	**11 tables**, 10 indexes
API endpoints	**38 routes + 1 health check = 39**

---

2. Kaunsi languages use hui hain

Language	Kahan	Kaam	Approx size
JavaScript (ES Modules)	`server/src/`	API, workflow, database queries, auth	24 files, ~4,670 lines
JavaScript	`server/tests/`	Automated acceptance tests	1 file, ~413 lines
TypeScript (TSX)	`web/src/`	React pages aur components	20 files, ~7,690 lines
TypeScript (TS)	`web/src/lib`, hooks, config	API client, types, helpers	5 files, ~1,380 lines
SQL	`server/src/` ke andar	Schema, indexes, queries	JavaScript strings ke roop mein
CSS3	`web/src/styles.css`	Poora design system	1 file, ~3,160 lines
HTML5	`web/index.html`	SPA ka entry page	1 file, 23 lines
JSON	package/config files	Build aur dependency config	6 files
Markdown	`README.md`, `DESIGN.md`, guides	Documentation	4 files

> **Important:** SQL alag `.sql` files mein nahi hai. Schema aur queries JavaScript ke andar template strings aur prepared statements ke roop mein likhi gayi hain. Isse project ko alag database client ki zaroorat nahi padti.

Backend JavaScript hai, TypeScript nahi. Iska matlab server ke liye koi compile step nahi chahiye — `node server/src/index.js` seedha chalta hai. Input validation **Zod** se hoti hai, isliye galat data API tak pahunchta hi nahi.

3. Frontend — kya use hua aur kyun

Technology	Version	Kaam
React	19.x	UI components
React DOM	19.x	Browser mein render karna
React Router DOM	7.x	Pages, protected routes, role-based guards
TypeScript	5.x	Type safety aur autocomplete
Vite	6.x	Dev server aur production build
lucide-react	0.468	Icons
Hand-written CSS	—	Design system, responsive layout, print styles

Frontend ka structure

- **Pages:** Dashboard, Tasks, Task Detail, Action Center, Team, Reports, Meetings,

Meeting Detail, AI Insights, Administration, Login.

- **Components:** layout, dialogs, task cards, timeline, shared UI.
- **State management:** Redux nahi. React **Context API** (Auth + Toast) aur ek custom

`useApi` hook — loading, error aur reload handle karta hai.

- **API calls:** ek central `lib/api.ts` wrapper, native `fetch` se. Har request ke

saath Bearer token jata hai.

- **Charts:** koi chart library nahi — simple CSS bars use hue hain, taaki bundle

halka rahe.

- **Styling:** Tailwind, Bootstrap ya koi UI kit nahi. Poora CSS hand-written hai.

Development vs production

- **Development:** Vite `:5173` par chalta hai aur `/api` requests ko Express `:4000`

par proxy karta hai. Isse CORS ki zaroorat nahi padti.

- **Production:** `npm run build` se `web/dist` banta hai, jise Express hi serve karta

hai. UI aur API **ek hi URL** par chalte hain.

- **Split deploy:** agar UI Vercel par ho, to `VITE_API_URL` se API ka poora address

diya jata hai. Ye optional hai.

---

4. Backend — kya use hua aur kyun

Technology	Version	Kaam
Node.js	24.x (minimum 22.5)	Runtime
Express	5.x	HTTP server, routing, middleware
Zod	3.x	Request body validation
cors	2.x	Allowed browser origins
node:sqlite	Node built-in	Database driver
node:crypto	Node built-in	Password hashing aur token signing

Backend ke main modules

File	Zimmedari
`index.js`	Server start, database open, seed, port
`app.js`	Express app, middleware, routes mount, static UI
`db.js`	SQLite connection, schema, indexes
`auth.js`	Password hashing, token sign/verify, login rate limit
`rbac.js`	Role aur team ke hisaab se data scope
`workflow.js`	Task lifecycle, approvals, extensions, ratings, groups
`reports.js`	Assigned vs self-created reports, team metrics
`ai.js`	AI calls, heuristic fallback, workload insights
`seed.js`	Demo organisation aur demo data
`repo.js`	Common database queries
`dashboard.js`	Dashboard ka combined data
`notify.js`	In-app notifications
`routes/`	10 route files — HTTP endpoints

Backend design ke important points

- **Koi ORM nahi** (Prisma/Sequelize nahi). Raw SQL + prepared statements.
- **Koi compile/build step nahi** backend ke liye.
- **Synchronous database calls** — `DatabaseSync` use hota hai.
- **Transactions** multi-step actions mein: `BEGIN IMMEDIATE` → changes → `COMMIT`,

error par `ROLLBACK`.

- **Validation** har route par Zod se.
- **Error handling** ek central middleware se — client ko safe message, server log mein

poora error.

- **Permissions server par** — UI ke buttons sirf presentation hain; asli decision API

leta hai.

---

5. Database — SQLite aur connection kaise hota hai

5.1 Database kya hai

Detail	Value
Engine	**SQLite 3**
Driver	Node ka built-in `node:sqlite` (`DatabaseSync`)
File	`server/data/meridian.db` (default)
Mode	**WAL** (Write-Ahead Logging)
Foreign keys	ON
Tables	**11**
Indexes	**10**
External database server	**Nahi**
Native package / compile	**Nahi**

> SQLite ek **embedded database** hai. Iska matlab database server alag se nahi chalta — database ek file hai, aur application us file ko directly open karti hai. Isliye username/password, host ya port ki zaroorat nahi padti.

5.2 Connection kaise khulta hai — step by step

`server/src/db.js` ka `openDatabase()` function ye kaam karta hai:

1. **Path tay hota hai:** `process.env.DB_PATH` ya default `server/data/meridian.db`.
2. **Folder banaya jata hai:** agar `server/data/` nahi hai to `mkdirSync` se ban jata

hai.

1. **File open hoti hai:** `new DatabaseSync(dbPath)` — yahi "connection" hai.
2. **Foreign keys on:** `PRAGMA foreign_keys = ON;`
3. **WAL mode on:** `PRAGMA journal_mode = WAL;` — faster reads aur better concurrency.

Kuch filesystems par WAL available nahi hota, isliye error safely ignore hota hai.

1. **Schema apply hota hai:** `CREATE TABLE IF NOT EXISTS ...` aur `CREATE INDEX IF NOT

EXISTS ...` — pehli baar chalane par tables khud ban jate hain.

1. **Seed:** agar database khaali hai, `seedDemo()` demo organisation, users, tasks aur

meetings bana deta hai.

1. **App ko database milta hai:** `createApp(db)` — route modules ko wahi database

object pass hota hai.

Code ka asli shape:

```
export function openDatabase(dbPath = process.env.DB_PATH || DEFAULT_DB_PATH) {
  fs.mkdirSync(path.dirname(dbPath), { recursive: true });
  const db = new DatabaseSync(dbPath);
  db.exec('PRAGMA foreign_keys = ON;');
  db.exec('PRAGMA journal_mode = WAL;');
  db.exec(SCHEMA);
  return db;
}
```

5.3 Query kaise chalti hai

Har query prepared statement se chalti hai:

```
const rows = db.prepare('SELECT * FROM tasks WHERE assignee_id = ?').all(userId);
const task = db.prepare('SELECT * FROM tasks WHERE id = ?').get(taskId);
db.prepare('UPDATE tasks SET status = ? WHERE id = ?').run(status, taskId);
```

Isse teen fayde hain:

- **SQL injection safe** — values alag se bind hoti hain.
- **Fast** — statement reuse hota hai.

- **Simple** — koi ORM mapping nahi.

5.4 Tables ka kaam

Table	Kaam
`departments`	Departments aur unke senior manager
`users`	4 roles, manager, department, password hash
`tasks`	Task, status, priority, approval chain, dates
`group_members`	Group task ke members aur leader
`task_events`	Permanent audit trail — har action ka record
`task_comments`	Task par discussion
`extension_requests`	Due date extension ka two-level approval
`ratings`	Manager aur senior ki alag 1–5 ratings
`meetings`	Transcript, AI summary, decisions, action items
`notifications`	In-app alerts
`settings`	AI configuration aur API key

5.5 Relations (simple)

- ek **department** ke andar kai **users**
- ek **user** kai **tasks** assign kar sakta hai ya bana sakta hai
- ek **task** ke kai **events**, **comments**, **extensions** aur **ratings**
- ek **group task** ke andar kai **group_members** aur **group parts** (child tasks)
- ek **meeting** ke action items se naye **tasks** ban sakte hain

5.6 10 indexes

Indexes in columns par hain jo sabse zyada filter hote hain:

`assignee_id`, `manager_id`, `senior_manager_id`, `department_id`, `status`, `parent_task_id`, `task_events.task_id`, `task_comments.task_id`, `notifications(user_id, is_read)`,

`meetings.department_id`.

Isse list, dashboard aur approval queries fast rehti hain.

5.7 Database path har environment mein

Environment	`DB_PATH`	Behaviour
Local development	default	`server/data/meridian.db`
Tests	`:memory:`	Temporary in-memory database, file nahi banti
Render free	`/tmp/meridian.db`	Temporary; restart par demo data reset
Railway	`/data/meridian.db`	Volume par permanent
Local production check	`%TEMP%\meridian.db`	Test ke liye safe

5.8 SQLite kyun chuna gaya

- **Zero setup** — koi database server install nahi.
- **Zero cost** — koi managed database plan nahi.
- **Portable** — poora data ek file mein; copy/backup aasan.
- **Fast** — local file, network round-trip nahi.
- **Demo ke liye perfect** — 10–50 logon ki team aaram se chala sakti hai.

5.9 SQLite ki limits (honest note)

- Ek waqt mein **ek hi writer** — bahut zyada parallel writes par wait hota hai.
- Horizontal scaling ke liye theek nahi — multiple servers ek file share nahi kar sakte.
- Bade organisation (hazaron users) ke liye **PostgreSQL** better rahega.
- Schema migrations ka koi formal system nahi — sirf `CREATE TABLE IF NOT EXISTS`.
- Data file ko share nahi karna chahiye, kyunki usme password hashes aur AI key hote hain.

---

6. Architecture — request ka safar

6.1 Development

```
Browser (React + Vite :5173)
        │
        │  /api request (proxy)
        ▼
Express API (:4000)
        │
        ▼
node:sqlite  ──►  server/data/meridian.db
```

6.2 Production (single service)

```
Browser
   │  HTTPS + JSON + Bearer token
   ▼
Express (:PORT)
   ├─ /api/*   → API routes → workflow → database
   └─ /*       → web/dist (React build)
        │
        ▼
   SQLite file (DB_PATH)
```

Isi wajah se manager ko sirf **ek URL** dena padta hai — UI aur API alag nahi dikhte.

6.3 Ek API request ka flow

1. Browser `fetch` se request bhejta hai, saath mein Bearer token.
2. Express route match karta hai.
3. Auth middleware token verify karta hai aur user nikalta hai.
4. Zod request body validate karta hai.
5. RBAC check hota hai — user is data/action ka haqdaar hai ya nahi.
6. Workflow function business rule lagata hai (status, approval level, etc.).
7. Prepared statement database par chalti hai.
8. Audit event aur notification likhe jate hain.

9. JSON response browser ko jata hai.

---

7. API — 39 endpoints

Area	Endpoints	Kaam
Authentication	4	Login, current user, change password, session check
Users	2	Directory aur team list (scope ke andar)
Tasks	9	List, detail, create, edit, actions, comments, decision, extension, rating
Approvals	1	Pending approvals ka combined queue
Reports	2	Person report aur team report
Meetings	5	List, detail, create, AI process, action item se task
AI	1	Workload aur due-date insights
Notifications	3	List, mark one read, mark all read
Dashboard	1	Stats, action queue, focus, team pulse
Administration	10	Users, departments, AI settings, password reset
Health	1	`/api/health` — deployment health check

Total: 38 business routes + 1 health check = 39.

---

8. Authentication aur security

Layer	Kaise
Password storage	`scrypt` + random 16-byte salt, 64-byte hash
Password compare	`crypto.timingSafeEqual` (timing attack safe)
Session	HMAC-SHA256 signed token, 7 din expiry
Token transport	`Authorization: Bearer <token>`
Login protection	8 galat attempts / 15 minute per IP + email
Authorization	Server-side RBAC har route par
CORS	`CORS_ORIGIN` se sirf allowed origins
Security headers	`nosniff`, `X-Frame-Options: DENY`, referrer aur permissions policy
Audit	Har status change, comment aur decision `task_events` mein
Admin boundary	Admin workflow actions nahi kar sakta

> Passwords kabhi API se wapas nahi bheje jate. AI key sirf masked hint ke roop mein dikhti hai.

9. Roles aur permissions

Capability	Employee	Manager	Senior Manager	Admin
Apne tasks	Haan	Haan	Haan	Read-only
Assign karna	Khud ko	Direct reports	Department	Nahi
Activation approve	Nahi	Haan	Apni line	Nahi
Completion review	Nahi	Level 1	Level 2	Nahi
Team oversight	Nahi	Direct reports	Department	Sab
Reports	Apne	Apne + reports	Department	Organisation
Meetings	Apni/participant	Apne + reports	Department	Sab
Raw transcript	Apni banayi	Creator/participant ke manager	Department	Sab
Users/departments/AI	Nahi	Nahi	Nahi	Haan

Ye rules **sirf UI mein nahi** — API har request par khud check karta hai. URL badal kar bhi doosri team ka data nahi khul sakta.

---

10. Core workflow — technical summary

- **Task create:** employee apne liye banaye to `PENDING_CREATION_APPROVAL`; manager

kisi ko assign kare to seedha `ACTIVE`.

- **Activation:** sirf assigned manager approve/reject karega; reject par reason

compulsory; creator revise karke resubmit karega.

- **Start:** sirf assignee, sirf `ACTIVE` se.
- **Completion:** `ACTIVE / IN_PROGRESS / REWORK` se submit → `PENDING_MANAGER_REVIEW` →

manager approve → `PENDING_SENIOR_REVIEW` → senior approve → `COMPLETED`.

- **Manager reject:** `REWORK` (assignee ke paas).
- **Senior reject:** `RETURNED_TO_MANAGER`; manager `REQUEST_CORRECTIONS` ya

`RESUBMIT_TO_SENIOR` chune.

- **Extension:** sirf assignee, kaam ke dauraan, naya date purane se baad ka; manager →

senior; dono approve par due date update.

- **Ratings:** sirf `COMPLETED` par, sirf assigned manager/senior, 1–5 + remarks.
- **Cancel:** creator ya assigned manager, reason ke saath.
- **Group Mode A:** sab contribution tasks complete hone par hi consolidated submit.
- **Group Mode B:** leader kabhi bhi submit kar sakta hai; contributions track hoti hain.

Poora user-facing explanation `USER_GUIDE.md` mein hai.

---

11. AI integration

- **Provider:** OpenAI-compatible Chat Completions API.
- **Default endpoint:** `https://api.openai.com/v1`
- **Default model:** `gpt-4o-mini`
- **Temperature:** `0.2` (stable, factual output)
- **Call:** native `fetch` — koi AI SDK nahi.
- **Configuration:** Admin → AI settings (DB mein save).
- **Key security:** API se sirf masked hint; raw key kabhi return nahi.
- **Fallback:** key na ho ya provider fail ho, to built-in **heuristic** extractor chalta

hai. UI mein source label dikhta hai:

- **Scope:** AI insights sirf us data se bante hain jo user dekh sakta hai.

AI meeting se **summary, decisions aur action items** nikalta hai. Har action item ek click mein governed task ban sakta hai.

---

12. Reports aur insights

- **Assigned work:** creator ≠ assignee — manager ne assign kiya.
- **Self-created work:** creator = assignee — employee ne khud banaya.
- **Combined:** dono ka total, reference ke liye.
- **Metrics:** total, completed, open, overdue, on-time rate, average cycle days,

average effort hours, manager rating, senior rating.

- **CSV export:** browser mein generate hota hai.
- **Scope:** employee apne, manager apne + reports, senior department, admin

organisation.

13. Project structure

```
Default Project/
├─ package.json           Monorepo root + scripts
├─ scripts/
│  ├─ dev.js              API + Vite dono start
│  └─ build-docs.mjs      Markdown docs → HTML
├─ server/
│  ├─ src/                24 JavaScript files
│  │  ├─ index.js         Entry point (:4000)
│  │  ├─ app.js           Express app + static UI
│  │  ├─ db.js            SQLite connection + schema
│  │  ├─ auth.js          Hashing + tokens
│  │  ├─ rbac.js          Role/team scoping
│  │  ├─ workflow.js      Task lifecycle (largest module)
│  │  ├─ reports.js       Split reports
│  │  ├─ ai.js            AI + fallback
│  │  └─ routes/          10 route files
│  ├─ tests/              12 acceptance scenarios
│  └─ package.json
├─ web/
│  ├─ src/
│  │  ├─ pages/           11 pages
│  │  ├─ components/      Shared UI
│  │  ├─ context/         Auth + Toast
│  │  ├─ hooks/           useApi
│  │  ├─ lib/             api.ts, types, constants, format
│  │  ├─ App.tsx          Routing + role guards
│  │  └─ styles.css       Design system
│  ├─ index.html
│  ├─ vite.config.ts
│  └─ package.json
├─ docs/                  Printable HTML + PDF guides
├─ Dockerfile             Multi-stage production image
├─ render.yaml            Render blueprint
├─ railway.json           Railway health check
└─ DEPLOY.md              Hosting guide
```

14. Dependencies — sirf 12 direct packages

Backend (3): `express`, `zod`, `cors`

Frontend runtime (4): `react`, `react-dom`, `react-router-dom`, `lucide-react`

Frontend dev (5): `vite`, `typescript`, `@vitejs/plugin-react`, `@types/react`, `@types/react-dom`

Koi Docker dependency, MongoDB, MySQL, Prisma, Tailwind, Redux ya Python use nahi hua. `node_modules` ka size approx **101 MB** hai.

---

15. Build, run, test aur deploy

Kaam	Command
Install	`npm install`
Demo data seed	`npm run seed`
Dev (API + UI)	`npm run dev`
Sirf API	`npm run dev:server`
Sirf UI	`npm run dev:web`
Production build	`npm run build`
Production run	`npm start`
Tests (12)	`npm test`
Type check	`npx tsc -p web/tsconfig.json --noEmit`
Docs build	`npm run docs`

Deploy kaise hota hai

1. **Dockerfile** multi-stage build karta hai:
   - Stage 1: React UI build (`web/dist`).

- Stage 2: sirf server dependencies + built UI.

2. Container `node server/src/index.js` chalata hai.
3. Express `$PORT` par UI aur `/api` dono serve karta hai.
4. Health check: `/api/health`.
5. **Render free:** `/tmp/meridian.db` — data restart par reset.
6. **Railway:** `/data/meridian.db` — volume ke saath permanent.
7. **Vercel:** sirf UI host ho sakti hai; SQLite Vercel Functions par persist nahi hota.

---

16. Testing aur quality

- **Runner:** Node ka built-in `node:test` (Jest/Vitest nahi).
- **Assertions:** `node:assert/strict`.
- **Database:** in-memory SQLite — test ke baad kuch nahi bachta.
- **Scenarios:** 12.

Cover kiya gaya:

1. Demo accounts authenticate hote hain.
2. Self-created task activation ke bina start nahi hota; self-approval block hai.
3. Completion manager → senior order follow karta hai.
4. Senior rejection manager ke paas jata hai, employee ke paas nahi.
5. Extension dono approval ke baad hi due date badalti hai.
6. Group Mode A ke parts isolated hain aur sab complete hone chahiye.
7. Group Mode B mein leader-only submission allowed hai.
8. Reports assigned aur self-created ko alag rakhte hain.
9. Transcript role/team ke hisaab se restricted hai.
10. Admin platform manage karta hai lekin workflow mein nahi aata.
11. Manager apni team ke bahar ka task nahi padh sakta.
12. AI insights authorised scope se bahar nahi jate.

Frontend ke liye TypeScript type-check (`tsc --noEmit`) aur production build (`vite build`) chalaye jate hain.

---

17. Important design decisions

1. **Server-authoritative permissions** — UI button chhupana security nahi hai; API

har request par decide karta hai.

1. **Activation gate** — employee ka self-created kaam manager ki approval ke bina

shuru nahi ho sakta.

1. **Two-tier completion** — manager aur senior, dono zaroori; koi level skip nahi.
2. **Senior rejection → manager** — employee ko seedha wapas nahi bheja jata.
3. **Due date immutable** — sirf extension workflow se badalti hai.
4. **Separate ratings** — manager aur senior ki rating alag records hain.
5. **Split reporting** — assigned aur self-created work kabhi mix nahi hote.
6. **Transcript privacy** — raw text sirf authorised roles ko.
7. **Admin read-only** — platform management aur workflow alag rakhe gaye.
8. **AI optional** — key na ho to deterministic fallback, feature band nahi hota.
9. **Zero native dependencies** — `node:sqlite` se install/compile simple.
10. **Single-service runtime** — UI + API ek URL par, deployment aasan.
11. **Audit trail** — har action ka permanent record.
12. **Monorepo, clean separation** — `server/` aur `web/` alag, lekin ek command se

chalte hain.

18. Limitations aur future scope

Limitation	Asar	Future fix
SQLite single writer	Bahut zyada parallel writes slow	PostgreSQL/Neon par migrate
No migration framework	Schema changes manual	Prisma/Drizzle migrations
Backend JavaScript	Runtime type safety kam	TypeScript backend
In-memory rate limit	Restart par reset, per-instance	Redis-based limiter
In-app notifications only	Email/SMS nahi	Email/push service
AI external dependency	Provider down ho to fallback	Local model option
Free hosting ephemeral disk	Data reset ho sakta hai	Railway volume / managed DB
No file attachments	Task ke saath files nahi	Object storage (S3/R2)

---

19. Manager ke common sawal

Q: Project kaunsi language mein bana hai? Frontend TypeScript/React mein, backend JavaScript/Node.js mein, aur database SQL (SQLite) hai.

Q: Frontend aur backend alag hain ya ek? Code alag folders (`web/` aur `server/`) mein hai, lekin production mein ek hi service dono ko serve karti hai — user ko ek URL dikhta hai.

Q: Database connection kaise hota hai? SQLite embedded hai. `DB_PATH` se file ka path milta hai, `new DatabaseSync(path)` se file open hoti hai, phir schema aur PRAGMAs apply hote hain. Koi host, port ya password nahi.

Q: ORM kyun nahi use kiya? Project chhota aur clear rakhne ke liye. Raw SQL + prepared statements safe aur fast hain, aur extra dependency nahi chahiye.

Q: Data kitna secure hai? Passwords scrypt se hashed, tokens HMAC-signed, har request par RBAC, audit trail, aur sensitive keys server-side.

Q: AI kaun sa use hua hai? OpenAI-compatible API, default `gpt-4o-mini`. Key na ho to built-in analysis chalta hai.

Q: Kitne users ke liye theek hai? Demo aur chhoti/medium team (approx 10–50 users) ke liye achha. Bade scale ke liye PostgreSQL par shift karna chahiye.

Q: Hosting ka kya setup hai? Docker image ek hi port par UI + API chalati hai. Render par free demo, Railway par persistent volume ke saath.

Q: Tests hain? Haan — 12 automated scenarios, in-memory database par.

---

20. Glossary — technical shabd simple mein

Shabd	Matlab
Frontend	Jo user browser mein dekhta hai
Backend	Server par chalne wala logic aur API
API	Frontend aur backend ke beech baat karne ka rasta
Database	Data save karne ki jagah
SQL	Database se baat karne ki language
ORM	Database ko objects ki tarah use karne ki library
Migration	Database schema ko safely badalne ka process
Token	Login ke baad mila hua signed pass
Hash	Password ko irreversible form mein badalna
RBAC	Role ke hisaab se access control
WAL	SQLite ka fast write mode
Docker	App ko ek portable container mein pack karna
Environment variable	Server ki configuration value (jaise `DB_PATH`)
Seed	Demo data automatically bharna
Health check	Server zinda hai ya nahi, ye batane wala endpoint

21. Ek line mein

> **React + TypeScript frontend, Node + Express JavaScript backend, SQLite embedded database, raw SQL + prepared statements, aur optional AI — sab ek hi service mein deploy ho jata hai.**